SMART INDIA HACKATHON · PROBLEM STATEMENT 26107

BIS Sahayak Architecture

An AI assistant for Indian Standards and Bureau of Indian Standards services that answers in plain language and shows its sources — with every citation checked in code before it reaches the screen.

6,209	220	25	66
Indian Standards indexed & embedded	document passages with clause locators	priority product groups → 82 committees	backend tests passing

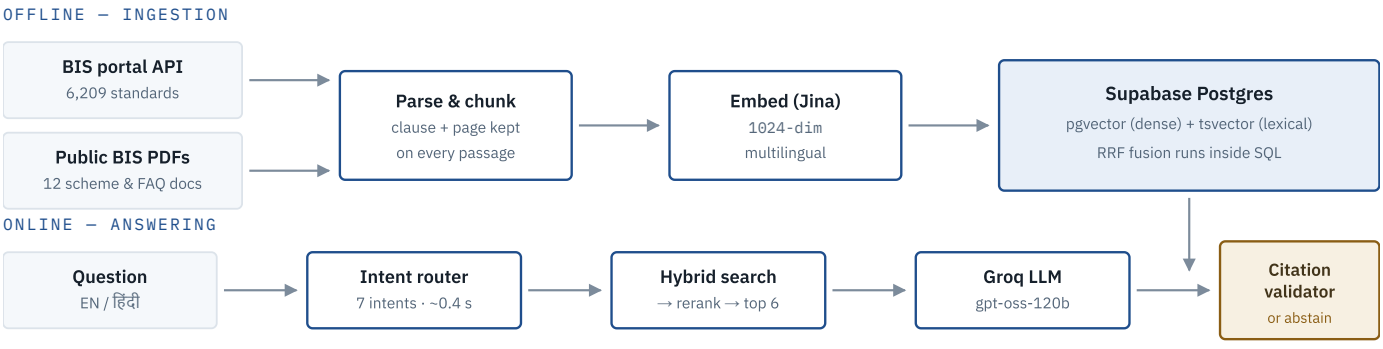
01 The problem, and the shape of the answer

BIS publishes thousands of standards and runs a dozen services across separate portals and PDFs. An MSME asking *"I manufacture LED bulbs — which standards apply and what licence do I need?"* must search a catalogue, a scheme portal, a Quality Control Order and a lab directory independently. Students and consumers fare worse.

A general chatbot answers this fluently and sometimes wrongly. In a regulatory setting a confident wrong answer about a certification duty costs a small manufacturer real money. So the system is built around one constraint: **an answer must be traceable to a document, or the assistant declines to give it.**

> **The design thesis.** Retrieval quality gets the right passages in front of the model. Guardrails decide whether what comes back is allowed to be shown. Those are separate problems, and the second one is where most RAG demos quietly fail.

02 System at a glance



Every passage carries its own source URL, clause and page — so a citation resolves to an exact location, not just a document.

03 Backend — building the corpus

The BIS portal has no public documentation. The endpoints were found by reading the Angular bundle the catalogue site ships, then verified against known standards.

1 **Scope to 25 product groups, not all 15,000 standards**

The QCO-heavy consumer and industrial categories where BIS questions actually cluster — cement, cables, toys, hallmarking, helmets, LPG appliances — mapped onto **82 BIS sectional committees** (CED 2, ETD 9, MTD 10 …), which is BIS's own working taxonomy.

2 **The mapping verifies itself**

Each group declares known-important IS numbers. Ingestion fails loudly if a group's own committees don't contain them. This caught three non-obvious classifications: BIS files `IS 4151` motorcycle helmets under **CED 22 "Fire Fighting"**; electric toys `IS 15644` under ETD 32, not the Toys committee; solar pumps `IS 17018` under MED 20.

3 **Parse documents so a citation can point somewhere exact**

PDFs are read page by page so each chunk keeps a real page number; DOCX files carry no pages, so `page` stays null rather than being guessed. BIS FAQs are Q&A rather than numbered clauses, so question headings are detected and **an answer is never split from its question**.

4 **Embed, resumably**

Both backfills select rows where `embedding IS NULL`, so an interrupted run costs nothing. Requests to BIS are throttled to 1/sec and cached on disk.

> **Undocumented-API traps worth a slide of their own.** `getStandardsBySectorId` looks like the catalogue but is a *recent-publications* view, and silently returns 10 rows unless three parameters are all supplied. `getWebsitePSTechDepartmentWise` takes a `typeSelected` flag where only **7** gives a committee's full list — `1` returns 96 rows for cement instead of 174, missing `IS 269` entirely. Standard numbers are inconsistently cased: the catalogue holds both `IS 2347:2023` and `Is 2347:2023`.

04 Backend — retrieval

Two arms, fused

Dense — pgvector cosine over Jina embeddings. Catches meaning: a Hindi question matches English source text.

Lexical — Postgres full-text search. Catches exact tokens dense search drifts past: `IS 15111`, `HUID`, `CRS`.

Fused with **Reciprocal Rank Fusion** inside SQL — not a weighted score sum, because cosine distance and `ts_rank_cd` are not on comparable scales, so any weighting would be arbitrary.

Two deliberate choices

The lexical index uses the `simple` configuration, which does no stemming — so standard numbers survive indexing intact. Stopwords are then stripped from the *query* only.

Terms are ORed, not ANDed: the lexical arm exists for recall, and RRF plus reranking supply precision. Under AND, *"gold hallmarking jewellery"* matched nothing, because no single title contains all three words.

Reranking, then a relevance floor

Hybrid search optimises for recall and happily returns 30 loosely-related passages. A small model scores the top 15 in one call and keeps the best 6. Candidates below a relevance floor are **dropped rather than padding the context** — handing the answer model a cement passage under a hallmarking question is an invitation to cite it.

05 Intent routing — one strategy per question type

The eight required capabilities are not one retrieval problem. A cheap model classifies the question and extracts entities in ~0.4 s.

INTENT	TOOL	WHAT IT SEARCHES
`recommend_standards`	catalogue vector search	Which standards apply to a described product
`standard_lookup`	exact prefix + passages	A named standard and anything discussing it
`certification`	passage search	ISI mark, CRS, FMCS, licensing procedure
`hallmarking`	passage search	HUID, purity, jeweller registration
`labs`	filtered table query	Recognised testing laboratories by state and scope
`consumer`	passage search	Complaints, ISI mark misuse, consumer rights
`smalltalk`	none	Answers directly, retrieves nothing

A regex pass unions with the model's extraction, so an IS number the model drops still reaches the lookup tool. Intents compose: a product question fuses catalogue results with scheme text, because *“which standard and what licence”* is genuinely two questions.

06 The trust layer — what makes this defensible

Prompting a model to cite its sources does not make the citations real. Four checks run in code, independent of what the model was told.

A Marker validation

The model cites evidence as `[S1]`, `[S2]`. Every marker is parsed and matched against the passages actually retrieved. **A marker that resolves to nothing is stripped before display** — a citation that looks authoritative and points nowhere is worse than none.

B Abstention on weak evidence

Decided from rerank scores *before* composing, using the mean of the top three so one lucky match cannot carry an empty result set. A weak-evidence question never reaches a model that would be tempted to fill the gap.

C Clause-claim grounding

A clause number asserted in the answer but present in no retrieved passage is almost certainly invented. Flagged and reported.

D Scope discipline

Full IS texts are paywalled and deliberately absent. For a standard held only as catalogue metadata, the assistant states what it covers and links out — it must not quote clause text it has never seen. The UI labels these *“Catalogue metadata only.”*

> **Two failures found by running the system, not by tests.** The reranker caught only four specific exception types, so a provider rate limit propagated and killed whole requests — despite a fall-back path already written for exactly that case. Separately, the model emitted fullwidth CJK brackets `【S1】`, which matched no marker pattern: a confidently cited answer resolved to **zero sources**, with nothing to signal it.
>
> Both were failures *of the safety machinery itself.* A guardrail that silently stops working is worse than not having one — both now carry regression tests.

07 API reference

ENDPOINT	RETURNS
POST `/chat`	`answer, sources[], session_id, abstained, intent, latency_ms, structured, warnings[]`
POST `/chat/stream`	Server-Sent Events: `intent` → `token`* → `structured`? → `sources` → `done`. The `done` event carries `abstained, dropped_markers, uncited`.
GET `/standards/search`	Hybrid search over the catalogue — `results[], total`. Params: `q`, `limit` (≤50)
GET `/standards/{is_number}`	One standard plus siblings from the same sectional committee
GET `/labs`	Recognised testing laboratories. Params: `state`, `scope`, `limit`

ENDPOINT	RETURNS
GET **/health**	Live status of Groq, Supabase and the embedding provider, plus row counts

The source object — the unit that makes citation work

FIELD	MEANING
`marker`	Matches the inline marker in the answer text, e.g. `S1`
`title`	Document name, e.g. *FAQs FMCS*
`locator`	Exact position — `clause Q10, p. 4`
`url`	Public BIS page the reader can open
`is_number`	Standard number where applicable
`doc_type`	`catalogue` · `scheme_guideline` · `qco` · `faq` · `consumer` · `hallmarking`

08 Frontend

Stack

Next.js 16 (App Router) · TypeScript · Tailwind CSS · 27 components across 14 routes. No state library — React state plus one typed API client.

Routes

`/chat` · `/standards` · `/standards/[slug]` · `/labs` · `/verify` · `/wizard` · `/documents` · `/amendments` · `/profile` · `/settings` · `/onboarding` · `/login` · `/signup`

How it connects

A single client module (`lib/api.ts`) holds every call, so response shapes live in one place. `NEXT_PUBLIC_API_URL` points at the backend; leaving it unset falls back to fixtures so the UI stays developable offline.

Streaming uses `fetch` rather than `EventSource`, because the endpoint is a POST with a JSON body. SSE frames are parsed from the byte stream, carrying partial lines across chunk boundaries.

Citations as a first-class UI element

Markers render as inline superscript pills; a sources panel below each answer shows title, locator and a link out. Abstention is rendered as a considered answer, not an error toast.

> **Why the catalogue browser talks to the API rather than a bundled list.** The original fixtures carried invented committee attributions — `IS 13252` labelled "ETD 35 (IT Equipment)" when BIS files it under **LITD 7**, and `IS 16046` as "ETD 42" rather than **ETD 11**. Confidently wrong regulatory metadata is precisely the failure the project exists to prevent, and no citation validator can catch it: the validator checks that a marker resolves to *a* source, not that the source is *true*.

09 Technology choices

ROLE	CHOICE	REASONING
Answering	`openai/gpt-oss-120b` (Groq)	Strongest available on Groq, 131k context
Fallback	`qwen/qwen3.8-27b` (Groq)	Deliberately a different family — one provider-side fault cannot disable both
Routing & reranking	`openai/gpt-oss-20b` (Groq)	~0.4 s classification with clean JSON output
Embeddings	`jina-embeddings-v3`, 1024-dim	Multilingual; free tier covers the whole corpus in ~16 minutes
Vector store	Supabase Postgres + pgvector	Dense and lexical search in one query, fused server-side
Backend	FastAPI · Python 3.11+	Streaming, typed request validation, generated API docs
Frontend	Next.js 16 · TypeScript	App Router, server components, typed end to end

> **Three constraints that shaped the stack.** Groq serves no embeddings endpoint — verified, not assumed: 14 models, none embedding, and `/v1/embeddings` returns 404. Gemini's free tier caps at **1,000 embedded items per day**, which would have taken a week for this corpus. And pgvector's HNSW index caps at 2,000 dimensions, so any embedding model must be truncated below that — and re-normalised, since truncation does not preserve unit length.

10 Status & honest gaps

Working

- DONE 6,209 standards indexed and embedded
- DONE 220 document passages with clause locators
- DONE Hybrid retrieval, reranking, abstention
- DONE Chat + streaming API, citation validation
- DONE Frontend connected to live backend
- DONE Cross-lingual retrieval — a Hindi query returns the right English standard

Not yet

- GAP **No hallmarking or HUID documents.** Those questions correctly abstain rather than answer — the clearest corpus gap, and hallmarking is named in the problem statement
- GAP Testing-laboratory directory is empty
- GAP Evaluation harness — recall@k, citation precision, refusal rate
- GAP Consumer-complaint source documents

Stating these plainly is deliberate. A system that knows the boundary of what it can support is the entire argument of the project; a demo that hides its gaps contradicts the thing it is demonstrating.